FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Jose Gabriel Valencia Collaborator: _____
Section:	AN43
Professor:	_____
Date:	November 3, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA4)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA4_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]
Rule-Based Reasoning Actions: - Turn on the aircon - Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

Completed Laboratory Work

Use Case and Knowledge Base

We implemented a classroom disaster-planning assistant for a local government unit. The new scenario is a typhoon with expected flooding in an urban area. RBR applies explicit planning rules, while CBR retrieves and adapts a similar stored case. All case descriptions, resource ratios, and recommended actions are illustrative exercise assumptions.

Part 1 Knowledge Representation

Task 1A is implemented by make_facts() and the FACTS dictionary. Eight facts are grouped into three categories. Strings represent categories, integers represent quantities or severity, and a Boolean represents expected flooding. Both reasoning methods read this same structured knowledge base.

Category	Fact	Sample value
Weather	hazard	typhoon
Weather	severity	5 on an exercise scale of 1 to 5
Weather	expected_flooding	True
Community	area_type	urban
Community	affected_people	1,200 people
Community	road_access	restricted
Resources	shelter_places	700 people
Resources	response_teams	5 teams

The complete executable source is included under Your Code Laboratory Tasks. The separate ta4_reasoning.py file contains the same code and requires only Python's standard library.

Part 2 Rule Based Reasoning

Task 2A IF THEN Rules

Function	IF condition	THEN action
rule_storm	Typhoon and severity at least 4	Prepare LGU coordination and public updates
rule_flood	Expected flooding is True	Review early evacuation from flood-prone areas
rule_shelter	Affected people exceed available shelter places	Identify the additional shelter places needed
rule_teams	Available teams are below one team per 200 people	Identify the additional response teams needed
rule_urban	Urban area and expected flooding	Coordinate evacuation support by neighborhood

Task 2B Inference Engine

infer() copies the facts into working memory and iterates through every function in RULES. Each function returns an action when its condition is true and None otherwise. The output loop prints all returned actions, so one matched rule does not stop later rules. This implementation performs a single pass over current facts; it does not claim to perform multi-step rule chaining.

Task 2C Observed Result

All five rules fired for the sample facts. The shelter rule compared all 1,200 affected people with 700 available places and identified a 500-place gap. The team rule calculated ceil(1200 / 200) = 6 teams and identified one additional team. These outputs follow the explicit exercise rules, not learned or official emergency-management ratios.

Run command: python ta4_reasoning.py --part rbr

Rule Based Reasoning Output

Screenshot of stdout captured from an actual Python execution and displayed in a browser.

RBR program output

```
python ta4_reasoning.py --part rbr

TA4 DISASTER PLANNING EXERCISE - illustrative data only
INPUT - 8 facts in 3 categories
  weather: {'hazard': 'typhoon', 'severity': 5, 'expected_flooding': True}
  community: {'area_type': 'urban', 'affected_people': 1200, 'road_access': 'restricted'}
  resources: {'shelter_places': 700, 'response_teams': 5}

RBR PROCESS - evaluate all 5 IF-THEN rules
  rule_storm: FIRED
  rule_flood: FIRED
  rule_shelter: FIRED
  rule_teams: FIRED
  rule_urban: FIRED
RBR OUTPUT - Rule-Based Reasoning Actions:
- Prepare LGU coordination and public updates.
- Review early evacuation from flood-prone areas.
- Review 500 additional shelter places for all affected people.
- Review 1 additional response team(s); rule total: 6.
- Coordinate evacuation support by urban neighborhood.

Demo used a temporary library; persistent saved cases were unchanged.
```

Figure 1 RBR input facts execution trace and inferred actions

The trace verifies that each of the five IF-THEN rules was checked. The output lists all applicable actions for the same knowledge base used by the CBR demonstration.

Part 3 Case Based Reasoning

Task 3A Case Base

seed_cases() creates four invented cases. Each case contains an ID, a problem using the same three-category fact structure, a matching solution, and a classroom-review note. The library includes two typhoon cases, one flood case, and one earthquake case.

Case	Hazard	Affected people	Stored shelter places	Stored teams
TYPHOON-01	Typhoon	1,000	800	6
TYPHOON-02	Typhoon	2,000	1,600	10
FLOOD-01	Flood	1,500	1,200	8
QUAKE-01	Earthquake	800	600	7

Retrieve Similar Cases

retrieve() filters by primary hazard and ranks the remaining cases using similarity(). The eight manually chosen weights sum to 1. Categorical and Boolean fields use exact matching; severity and road access use ordinal distance; population and resources use relative numerical similarity. The sample ranked TYPHOON-01 at 90.1% and TYPHOON-02 at 89.1%. Similarity expresses resemblance, not response success probability.

Tasks 3C and 3D Reuse and Revise

reuse() deep-copies the most similar case's solution and scales resource counts by the population ratio and a severity adjustment. The sample factors are 1200 / 1000 = 1.20 and 1 + 0.15 x (5 - 4) = 1.15. The adapted plan proposes 1,104 shelter places and 9 teams. revise() accepts manual corrections or keeps a field unchanged when Enter is pressed. The reproducible demo explicitly adds one coordination team, giving 10 teams.

Task 3E Retain

retain() saves the reviewed problem, solution, and review note as LEARNED-05 in JSON. The demo reloads that file: the case count increases from 4 to 5 and the same query retrieves the learned case at 100%. Interactive mode asks for confirmation and keeps the case file after restart. Invalid reviewed counts, blank notes, and duplicate problems are rejected.

Case Based Reasoning Output

Screenshot of stdout captured from an actual Python execution and displayed in a browser.

CBR program output

```
python ta4_reasoning.py --part cbr

TA4 DISASTER PLANNING EXERCISE - illustrative data only
INPUT - 8 facts in 3 categories
  weather: {'hazard': 'typhoon', 'severity': 5, 'expected_flooding': True}
  community: {'area_type': 'urban', 'affected_people': 1200, 'road_access': 'restricted'}
  resources: {'shelter_places': 700, 'response_teams': 5}

CBR RETRIEVE - 4 stored cases; same-hazard filter
  TYPHOON-01: 90.1%
  TYPHOON-02: 89.1%
CBR REUSE - adapt TYPHOON-01 to 1,200 affected people
  Shelter places: 1104; teams: 9
  Evacuation: Review early urban evacuation and accessible transport.
CBR REVISE - response teams 9 -> 10
  Reviewed shelter places: 1104
CBR RETAIN - saved LEARNED-05; case count 4 -> 5
  Same query after reload: LEARNED-05 at 100.0%
  Stored review: Classroom review: kept the shelter estimate and added one coordination
team.

Demo used a temporary library; persistent saved cases were unchanged.
```

Figure 2 CBR retrieval reuse revision retention and retrieval after reload

The default demonstration uses a temporary library. It demonstrates a genuine JSON write and reload without changing a user's persistent cases. For manual revision and persistent learning, run: python ta4_reasoning.py --part cbr --interactive

Part 4 Comparative Reflection

Which System Was Easier to Implement and Why

RBR was easier to implement in this laboratory because each planning condition could be written directly as an IF-THEN function. A short loop checked the rules and collected their actions. CBR required more steps: representing previous cases, choosing similarity weights, normalizing numerical features, adapting a solution, supporting revision, and saving the learned case.

Which Is More Flexible for Unstructured Problems

CBR is generally more flexible when a new situation resembles a past experience but does not fit a precise rule. It can reuse part of a previous solution and adapt it to changed conditions. However, this flexibility still depends on useful case features and a good similarity measure. Our implementation uses structured dictionaries, so unstructured descriptions would first need to be converted into comparable features.

Which Is Better for Continuously Changing Data

CBR is useful when new reviewed experiences and outcomes arrive regularly because they can be added to the case base without rewriting every rule. The laboratory demonstrates this by retrieving the newly retained case after reloading. RBR remains useful when changing facts must be checked against stable policies. If the policies themselves change, the rules must be maintained. Neither method guarantees that old knowledge remains correct.

Real World Applications of Both Methods

RBR can support equipment safety checks, eligibility screening under explicit policies, and appliance automation because the conditions and actions can be stated clearly. CBR can support technical troubleshooting, help-desk recommendations, and decision support that compares current situations with reviewed past cases. For disaster planning, rules can flag known resource gaps while similar cases provide candidate plans for human review.

Observed Difference on the Same Scenario

Our RBR output identified a 500-place shelter gap and one additional team under fixed exercise rules. CBR proposed 1,104 total shelter places and 9 teams from a prior plan, then retained a reviewed total of 10 teams. The difference comes from their knowledge sources: explicit rules versus adapted experience. It is not evidence that either result is an operationally correct emergency response.

Your Code Laboratory Tasks

ta4_reasoning.py Lines 1 to 44

```
"""TA4: knowledge representation, rule-based reasoning, and case-based reasoning.

All cases, response ratios, and actions are classroom examples, not emergency advice.
Run normally for a reproducible demo; add --interactive to revise and retain cases.
"""

import argparse
import json
import tempfile
from copy import deepcopy
from fractions import Fraction
from math import ceil
from pathlib import Path


# Task 1A: eight facts organized into three explicit categories.
def make_facts(hazard, severity, people, shelters, teams,
               flooding=True, area="urban", access="restricted"):
    return {
        "weather": {"hazard": hazard, "severity": severity,
                    "expected_flooding": flooding},
        "community": {"area_type": area, "affected_people": people,
                      "road_access": access},
        "resources": {"shelter_places": shelters, "response_teams": teams},
    }


FACTS = make_facts("typhoon", 5, 1200, 700, 5)


# Task 2A: each function represents one independently evaluated IF-THEN rule.
def rule_storm(facts):
    weather = facts["weather"]
    if weather["hazard"] == "typhoon" and weather["severity"] >= 4:
        return "Prepare LGU coordination and public updates."


def rule_flood(facts):
    if facts["weather"]["expected_flooding"]:
        return "Review early evacuation from flood-prone areas."


def rule_shelter(facts):
    gap = facts["community"]["affected_people"] - facts["resources"]["shelter_places"]
```

Your Code Continued

ta4_reasoning.py Lines 45 to 88

```
    if gap > 0:
        return f"Review {gap} additional shelter places for all affected people."


def rule_teams(facts):
    # One team per 200 people is an illustrative rule for this exercise only.
    required = ceil(facts["community"]["affected_people"] / 200)
    gap = required - facts["resources"]["response_teams"]
    if gap > 0:
        return f"Review {gap} additional response team(s); rule total: {required}."


def rule_urban(facts):
    if (facts["community"]["area_type"] == "urban"
            and facts["weather"]["expected_flooding"]):
        return "Coordinate evacuation support by urban neighborhood."


RULES = (rule_storm, rule_flood, rule_shelter, rule_teams, rule_urban)


# Task 2B: a single-pass inference engine checks every rule and collects all actions.
def infer(facts):
    working_memory = deepcopy(facts)
    return [(rule.__name__, rule(working_memory)) for rule in RULES]


# Task 3A: four invented cases, each with problem features and a solution.
def seed_cases():
    examples = [
        ("TYPHOON-01", make_facts("typhoon", 4, 1000, 600, 4), 800, 6),
        ("TYPHOON-02", make_facts("typhoon", 5, 2000, 1000, 8), 1600, 10),
        ("FLOOD-01", make_facts("flood", 4, 1500, 900, 5), 1200, 8),
        ("QUAKE-01", make_facts("earthquake", 4, 800, 400, 5,
                                flooding=False), 600, 7),
    ]
    return [{"id": name, "problem": problem, "solution": {
        "shelter_places": shelters, "response_teams": teams,
        "strategy": "Review accessible evacuation routes and shelter registration.",
    }, "outcome": "Synthetic classroom review; not a real disaster outcome."}
        for name, problem, shelters, teams in examples]


# Weights sum to 1. Numeric comparisons are normalized before weighting.
```

Your Code Continued

ta4_reasoning.py Lines 89 to 132

```
WEIGHTS = {"hazard": .20, "severity": .20, "expected_flooding": .10,
           "area_type": .10, "affected_people": .15, "road_access": .10,
           "shelter_places": .10, "response_teams": .05}


def similarity(new, stored):
    left = {key: value for group in new.values() for key, value in group.items()}
    right = {key: value for group in stored.values() for key, value in group.items()}
    score = 0.0
    for feature, weight in WEIGHTS.items():
        a, b = left[feature], right[feature]
        if feature == "severity":
            local = 1 - abs(a - b) / 4
        elif feature == "road_access":
            order = ("open", "restricted", "blocked")
            local = 1 - abs(order.index(a) - order.index(b)) / 2
        elif feature in ("affected_people", "shelter_places", "response_teams"):
            local = 1 - abs(a - b) / max(a, b, 1)
        else:
            local = float(a == b)
        score += weight * local
    return score


# Retrieve: compare only the same primary hazard and rank by similarity.
def retrieve(facts, cases):
    matches = [(similarity(facts, case["problem"]), case) for case in cases
               if case["problem"]["weather"]["hazard"] == facts["weather"]["hazard"]]
    return sorted(matches, key=lambda item: (-item[0], item[1]["id"]))


# Task 3C: reuse the closest solution, then adapt its resource counts.
def reuse(facts, case):
    if (facts["weather"]["hazard"] != case["problem"]["weather"]["hazard"]
            or similarity(facts, case["problem"]) < .65):
        raise ValueError("Reuse requires the same hazard and at least 65% similarity.")
    solution = deepcopy(case["solution"])
    old = case["problem"]
    scale = Fraction(facts["community"]["affected_people"],
                     old["community"]["affected_people"])
    difference = facts["weather"]["severity"] - old["weather"]["severity"]
    scale *= Fraction(100 + 15 * difference, 100)
    solution["shelter_places"] = min(facts["community"]["affected_people"],
                                     ceil(solution["shelter_places"] * scale))
```

Your Code Continued

ta4_reasoning.py Lines 133 to 176

```
    solution["response_teams"] = max(1, ceil(solution["response_teams"] * scale))
    if facts["weather"]["expected_flooding"]:
        area = facts["community"]["area_type"]
        solution["strategy"] = f"Review early {area} evacuation and accessible transport."
    return solution


# Task 3D: blank manual input keeps the proposed value unchanged.
def revise(solution, interactive=False):
    reviewed = deepcopy(solution)
    if interactive:
        for key in ("shelter_places", "response_teams"):
            value = input(f"Reviewed {key} [{reviewed[key]}]: ").strip()
            if value:
                reviewed[key] = int(value)
        value = input(f"Reviewed strategy [{reviewed['strategy']}]: ").strip()
        if value:
            reviewed["strategy"] = value
    else:
        # Explicit worked review, not an inferred action.
        reviewed["response_teams"] += 1
    return reviewed


# Task 3E: retain reviewed experience; JSON makes interactive learning persistent.
def retain(facts, solution, outcome, cases, path):
    people = facts["community"]["affected_people"]
    if (type(solution["shelter_places"]) is not int
            or not 0 <= solution["shelter_places"] <= people
            or type(solution["response_teams"]) is not int
            or solution["response_teams"] < 1
            or not solution["strategy"].strip() or not outcome.strip()):
        raise ValueError("Check reviewed resource counts, strategy, and outcome note.")
    if any(case["problem"] == facts for case in cases):
        raise ValueError("This exact problem is stored; no duplicate was added.")
    learned = {"id": f"LEARNED-{len(cases) + 1:02d}", "problem": deepcopy(facts),
               "solution": deepcopy(solution), "outcome": outcome}
    updated = [*cases, learned]
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(updated, indent=2) + "\n", encoding="utf-8")
    cases.append(learned)
    return learned
```

Your Code Continued

ta4_reasoning.py Lines 177 to 220

```
def run(part, interactive, path):
    facts = deepcopy(FACTS)
    print("TA4 DISASTER PLANNING EXERCISE - illustrative data only")
    print("INPUT - 8 facts in 3 categories")
    for category, values in facts.items():
        print(f"  {category}: {values}")
    if part in ("all", "rbr"):
        print("\nRBR PROCESS - evaluate all 5 IF-THEN rules")
        checked = infer(facts)
        for name, action in checked:
            print(f"  {name}: {'FIRED' if action else 'not applicable'}")
        print("RBR OUTPUT - Rule-Based Reasoning Actions:")
        for _, action in checked:
            if action:
                print("- " + action)
    if part in ("all", "cbr"):
        cases = (json.loads(path.read_text(encoding="utf-8"))
                 if path.exists() else seed_cases())
        print(f"\nCBR RETRIEVE - {len(cases)} stored cases; same-hazard filter")
        matches = retrieve(facts, cases)
        for score, case in matches:
            print(f"  {case['id']}: {score:.1%}")
        if not matches or matches[0][0] < .65:
            print("No suitable case for automatic reuse.")
            return
        best = matches[0][1]
        proposal = reuse(facts, best)
        people = facts["community"]["affected_people"]
        print(f"CBR REUSE - adapt {best['id']} to {people:,} affected people")
        print(f"  Shelter places: {proposal['shelter_places']}; "
              f"teams: {proposal['response_teams']}")
        print("  Evacuation: " + proposal["strategy"])
        reviewed = revise(proposal, interactive)
        print(f"CBR REVISE - response teams {proposal['response_teams']} "
              f"-> {reviewed['response_teams']}")
        print(f"  Reviewed shelter places: {reviewed['shelter_places']}")
        note = (input("Outcome or review note: ").strip() if interactive
                else "Classroom review: kept the shelter estimate "
                     "and added one coordination team.")
        if interactive and input("Retain this reviewed case? [y/N]: ").lower() != "y":
            print("Case was not retained.")
            return
        before = len(cases)
        learned = retain(facts, reviewed, note, cases, path)
```

Your Code Continued

ta4_reasoning.py Lines 221 to 251

```
        reloaded = json.loads(path.read_text(encoding="utf-8"))
        nearest = retrieve(facts, reloaded)[0]
        print(f"CBR RETAIN - saved {learned['id']}; "
              f"case count {before} -> {len(reloaded)}")
        print(f"  Same query after reload: {nearest[1]['id']} at {nearest[0]:.1%}")
        print(f"  Stored review: {nearest[1]['outcome']}")


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--part", choices=("all", "rbr", "cbr"), default="all")
    parser.add_argument("--interactive", action="store_true")
    parser.add_argument("--data-file", type=Path,
                        default=Path(__file__).with_name("cases.json"))
    args = parser.parse_args()
    try:
        if args.interactive:
            run(args.part, True, args.data_file)
        else:
            with tempfile.TemporaryDirectory(prefix="ta4-demo-") as directory:
                run(args.part, False, Path(directory) / "cases.json")
            print("\nDemo used a temporary library; "
                  "persistent saved cases were unchanged.")
    except (ValueError, OSError, EOFError, KeyboardInterrupt) as error:
        print(f"Stopped: {error}")
        return 1
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```